Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Sergio Arévalo Garzón

Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Libro (como el ejercicio Producto)

Libro
- titulo : String - autor : String - precio : double - ejemplares : int
+ Libro() + Libro(titulo: String, autor: String, precio: double, ejemplares: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + venderEjemplares(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna precio * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna precio - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **venderEjemplares(cantidad)**: si cantidad es menor o igual a ejemplares, resta cantidad a ejemplares y retorna "Operación realizada. Quedan X". Si no, no cambia ejemplares y retorna "No hay suficientes ejemplares".

Main

- Crear dos objetos: uno con new Libro() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar venderEjemplares dos veces: una con cantidad suficiente y otra con una cantidad mayor a ejemplares.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Libro.java** y **MainLibro.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String titulo;
private String autor;
private double precio;
private int ejemplares;

public Libro() {
}

public Libro(String titulo, String autor, double precio, int ejemplares) {
    this.titulo = titulo;
    this.autor = ____;
    this.____ = precio;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de titulo; escribe igual los de autor, precio y ejemplares, cambiando el tipo.

```
public String getTitulo() {
    return this.titulo;
}
public void setTitulo(String titulo) {
    this.titulo = titulo;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "Titulo: " + this.titulo + "\nAutor: " + this.autor + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.precio * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.precio - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. venderEjemplares. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia ejemplares.

```
public String venderEjemplares(int cantidad) {
    if (cantidad <= this.____) {
        this.ejemplares = this.ejemplares - ____;
        return "Operación realizada. Quedan " + this.ejemplares;
    } else {
        return "No hay suficientes ejemplares";
    }
}
```

5. Main. En otro archivo, MainLibro.java:

```
public class MainLibro {
    public static void main(String[] args) {
        Libro obj1 = new Libro();
        obj1.setTitulo("...");
        obj1.setAutor("...");
        obj1.setPrecio(____);
        obj1.setEjemplares(10);
        Libro obj2 = new Libro("...", "...", ____, ____);

        System.out.println(obj1.getTitulo());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.venderEjemplares(3));    // sí alcanza
        System.out.println(obj1.venderEjemplares(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Diego Alejandro Cely Ruiz

Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Boleta (como el ejercicio Producto)

Boleta
- evento : String - localidad : String - precio : double - disponibles : int
+ Boleta() + Boleta(evento: String, localidad: String, precio: double, disponibles: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + venderBoletas(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna precio * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna precio - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **venderBoletas(cantidad)**: si cantidad es menor o igual a disponibles, resta cantidad a disponibles y retorna "Operación realizada. Quedan X". Si no, no cambia disponibles y retorna "No hay suficientes disponibles".

Main

- Crear dos objetos: uno con new Boleta() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar venderBoletas dos veces: una con cantidad suficiente y otra con una cantidad mayor a disponibles.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Boleta.java** y **MainBoleta.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String evento;
private String localidad;
private double precio;
private int disponibles;

public Boleta() {
}

public Boleta(String evento, String localidad, double precio, int disponibles) {
    this.evento = evento;
    this.localidad = ____;
    this.____ = precio;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de evento; escribe igual los de localidad, precio y disponibles, cambiando el tipo.

```
public String getEvento() {
    return this.evento;
}
public void setEvento(String evento) {
    this.evento = evento;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "Evento: " + this.evento + "\nLocalidad: " + this.localidad + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.precio * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.precio - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. venderBoletas. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia disponibles.

```
public String venderBoletas(int cantidad) {
    if (cantidad <= this.____) {
        this.disponibles = this.disponibles - ____;
        return "Operación realizada. Quedan " + this.disponibles;
    } else {
        return "No hay suficientes disponibles";
    }
}
```

5. Main. En otro archivo, MainBoleta.java:

```
public class MainBoleta {
    public static void main(String[] args) {
        Boleta obj1 = new Boleta();
        obj1.setEvento("...");
        obj1.setLocalidad("...");
        obj1.setPrecio(____);
        obj1.setDisponibles(10);
        Boleta obj2 = new Boleta("...", "...", ____, ____);

        System.out.println(obj1.getEvento());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.venderBoletas(3));    // sí alcanza
        System.out.println(obj1.venderBoletas(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Rubén Felipe Gordo González
Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Bicicleta (como el ejercicio Producto)

Bicicleta
- marca : String - tipo : String - precioAlquiler : double - disponibles : int
+ Bicicleta() + Bicicleta(marca: String, tipo: String, precioAlquiler: double, disponibles: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + alquilar(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna precioAlquiler * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna precioAlquiler - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **alquilar(cantidad)**: si cantidad es menor o igual a disponibles, resta cantidad a disponibles y retorna "Operación realizada. Quedan X". Si no, no cambia disponibles y retorna "No hay suficientes disponibles".

Main

- Crear dos objetos: uno con new Bicicleta() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar alquilar dos veces: una con cantidad suficiente y otra con una cantidad mayor a disponibles.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Bicicleta.java** y **MainBicicleta.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String marca;
private String tipo;
private double precioAlquiler;
private int disponibles;

public Bicicleta() {
}

public Bicicleta(String marca, String tipo, double precioAlquiler, int disponibles) {
    this.marca = marca;
    this.tipo = ____;
    this.____ = precioAlquiler;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de marca; escribe igual los de tipo, precioAlquiler y disponibles, cambiando el tipo.

```
public String getMarca() {
    return this.marca;
}
public void setMarca(String marca) {
    this.marca = marca;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "Marca: " + this.marca + "\nTipo: " + this.tipo + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.precioAlquiler * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.precioAlquiler - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. alquilar. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia disponibles.

```
public String alquilar(int cantidad) {
    if (cantidad <= this.____) {
        this.disponibles = this.disponibles - ____;
        return "Operación realizada. Quedan " + this.disponibles;
    } else {
        return "No hay suficientes disponibles";
    }
}
```

5. Main. En otro archivo, MainBicicleta.java:

```
public class MainBicicleta {
    public static void main(String[] args) {
        Bicicleta obj1 = new Bicicleta();
        obj1.setMarca("...");
        obj1.setTipo("...");
        obj1.setPrecioAlquiler(____);
        obj1.setDisponibles(10);
        Bicicleta obj2 = new Bicicleta("...", "...", ____, ____);

        System.out.println(obj1.getMarca());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.alquilar(3));    // sí alcanza
        System.out.println(obj1.alquilar(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: José Luis Leguizamón Rojas
Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Curso (como el ejercicio Producto)

Curso
- nombreCurso : String - docente : String - valor : double - cupos : int
+ Curso() + Curso(nombreCurso: String, docente: String, valor: double, cupos: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + inscribirEstudiantes(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna valor * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna valor - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **inscribirEstudiantes(cantidad)**: si cantidad es menor o igual a cupos, resta cantidad a cupos y retorna "Operación realizada. Quedan X". Si no, no cambia cupos y retorna "No hay suficientes cupos".

Main

- Crear dos objetos: uno con new Curso() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar inscribirEstudiantes dos veces: una con cantidad suficiente y otra con una cantidad mayor a cupos.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Curso.java** y **MainCurso.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String nombreCurso;
private String docente;
private double valor;
private int cupos;

public Curso() {
}

public Curso(String nombreCurso, String docente, double valor, int cupos) {
    this.nombreCurso = nombreCurso;
    this.docente = ____;
    this.____ = valor;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de nombreCurso; escribe igual los de docente, valor y cupos, cambiando el tipo.

```
public String getNombreCurso() {
    return this.nombreCurso;
}
public void setNombreCurso(String nombreCurso) {
    this.nombreCurso = nombreCurso;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "NombreCurso: " + this.nombreCurso + "\nDocente: " + this.docente + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.valor * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.valor - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. inscribirEstudiantes. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia cupos.

```
public String inscribirEstudiantes(int cantidad) {
    if (cantidad <= this.____) {
        this.cupos = this.cupos - ____;
        return "Operación realizada. Quedan " + this.cupos;
    } else {
        return "No hay suficientes cupos";
    }
}
```

5. Main. En otro archivo, MainCurso.java:

```
public class MainCurso {
    public static void main(String[] args) {
        Curso obj1 = new Curso();
        obj1.setNombreCurso("...");
        obj1.setDocente("...");
        obj1.setValor(____);
        obj1.setCupos(10);
        Curso obj2 = new Curso("...", "...", ____, ____);

        System.out.println(obj1.getNombreCurso());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.inscribirEstudiantes(3));    // sí alcanza
        System.out.println(obj1.inscribirEstudiantes(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Ángel Fabián Madrigal Guzmán
Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Vuelo (como el ejercicio Producto)

Vuelo
- destino : String - aerolinea : String - valorTiquete : double - sillasDisponibles : int
+ Vuelo() + Vuelo(destino: String, aerolinea: String, valorTiquete: double, sillasDisponibles: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + reservarSillas(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna valorTiquete * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna valorTiquete - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **reservarSillas(cantidad)**: si cantidad es menor o igual a sillasDisponibles, resta cantidad a sillasDisponibles y retorna "Operación realizada. Quedan X". Si no, no cambia sillasDisponibles y retorna "No hay suficientes sillasDisponibles".

Main

- Crear dos objetos: uno con new Vuelo() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar reservarSillas dos veces: una con cantidad suficiente y otra con una cantidad mayor a sillasDisponibles.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Vuelo.java** y **MainVuelo.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String destino;
private String aerolinea;
private double valorTiquete;
private int sillasDisponibles;

public Vuelo() {
}

public Vuelo(String destino, String aerolinea, double valorTiquete, int sillasDisponibles) {
    this.destino = destino;
    this.aerolinea = ____;
    this.____ = valorTiquete;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de destino; escribe igual los de aerolinea, valorTiquete y sillasDisponibles, cambiando el tipo.

```
public String getDestino() {
    return this.destino;
}
public void setDestino(String destino) {
    this.destino = destino;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "Destino: " + this.destino + "\nAerolinea: " + this.aerolinea + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.valorTiquete * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.valorTiquete - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. reservarSillas. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia sillasDisponibles.

```
public String reservarSillas(int cantidad) {
    if (cantidad <= this.____) {
        this.sillasDisponibles = this.sillasDisponibles - ____;
        return "Operación realizada. Quedan " + this.sillasDisponibles;
    } else {
        return "No hay suficientes sillasDisponibles";
    }
}
```

5. Main. En otro archivo, MainVuelo.java:

```
public class MainVuelo {
    public static void main(String[] args) {
        Vuelo obj1 = new Vuelo();
        obj1.setDestino("...");
        obj1.setAerolinea("...");
        obj1.setValorTiquete(____);
        obj1.setSillasDisponibles(10);
        Vuelo obj2 = new Vuelo("...", "...", ____, ____);

        System.out.println(obj1.getDestino());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.reservarSillas(3));    // sí alcanza
        System.out.println(obj1.reservarSillas(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Camilo Andrés Páez Díaz

Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Celular (como el ejercicio Producto)

Celular
- marca : String - modelo : String - precio : double - unidades : int
+ Celular() + Celular(marca: String, modelo: String, precio: double, unidades: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + vender(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna precio * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna precio - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **vender(cantidad)**: si cantidad es menor o igual a unidades, resta cantidad a unidades y retorna "Operación realizada. Quedan X". Si no, no cambia unidades y retorna "No hay suficientes unidades".

Main

- Crear dos objetos: uno con new Celular() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar vender dos veces: una con cantidad suficiente y otra con una cantidad mayor a unidades.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Celular.java** y **MainCelular.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String marca;
private String modelo;
private double precio;
private int unidades;

public Celular() {
}

public Celular(String marca, String modelo, double precio, int unidades) {
    this.marca = marca;
    this.modelo = ____;
    this.____ = precio;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de marca; escribe igual los de modelo, precio y unidades, cambiando el tipo.

```
public String getMarca() {
    return this.marca;
}
public void setMarca(String marca) {
    this.marca = marca;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "Marca: " + this.marca + "\nModelo: " + this.modelo + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.precio * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.precio - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. vender. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia unidades.

```
public String vender(int cantidad) {
    if (cantidad <= this.____) {
        this.unidades = this.unidades - ____;
        return "Operación realizada. Quedan " + this.unidades;
    } else {
        return "No hay suficientes unidades";
    }
}
```

5. Main. En otro archivo, MainCelular.java:

```
public class MainCelular {
    public static void main(String[] args) {
        Celular obj1 = new Celular();
        obj1.setMarca("...");
        obj1.setModelo("...");
        obj1.setPrecio(____);
        obj1.setUnidades(10);
        Celular obj2 = new Celular("...", "...", ____, ____);

        System.out.println(obj1.getMarca());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.vender(3));    // sí alcanza
        System.out.println(obj1.vender(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Fabián David Sánchez Rojas

Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Plato (como el ejercicio Producto)

Plato
- nombre : String - tipoComida : String - precio : double - porciones : int
+ Plato() + Plato(nombre: String, tipoComida: String, precio: double, porciones: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + servirPorciones(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna precio * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna precio - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **servirPorciones(cantidad)**: si cantidad es menor o igual a porciones, resta cantidad a porciones y retorna "Operación realizada. Quedan X". Si no, no cambia porciones y retorna "No hay suficientes porciones".

Main

- Crear dos objetos: uno con new Plato() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar servirPorciones dos veces: una con cantidad suficiente y otra con una cantidad mayor a porciones.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Plato.java** y **MainPlato.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String nombre;
private String tipoComida;
private double precio;
private int porciones;

public Plato() {
}

public Plato(String nombre, String tipoComida, double precio, int porciones) {
    this.nombre = nombre;
    this.tipoComida = ____;
    this.____ = precio;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de nombre; escribe igual los de tipoComida, precio y porciones, cambiando el tipo.

```
public String getNombre() {
    return this.nombre;
}
public void setNombre(String nombre) {
    this.nombre = nombre;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "Nombre: " + this.nombre + "\nTipoComida: " + this.tipoComida + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.precio * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.precio - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. servirPorciones. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia porciones.

```
public String servirPorciones(int cantidad) {
    if (cantidad <= this.____) {
        this.porciones = this.porciones - ____;
        return "Operación realizada. Quedan " + this.porciones;
    } else {
        return "No hay suficientes porciones";
    }
}
```

5. Main. En otro archivo, MainPlato.java:

```
public class MainPlato {
    public static void main(String[] args) {
        Plato obj1 = new Plato();
        obj1.setNombre("...");
        obj1.setTipoComida("...");
        obj1.setPrecio(____);
        obj1.setPorciones(10);
        Plato obj2 = new Plato("...", "...", ____, ____);

        System.out.println(obj1.getNombre());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.servirPorciones(3));    // sí alcanza
        System.out.println(obj1.servirPorciones(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Iker Esteban Fuentes Torres

Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Vendedor (como el ejercicio Empleado)

Vendedor
- nombre : String - zona : String - salarioBase : double - ventasMes : int
+ Vendedor() + Vendedor(nombre: String, zona: String, salarioBase: double, ventasMes: int) + getters y setters de los 4 atributos + verDetalle() : String + visitarClientes(clientes: int) : String + registrarVenta() : String + visitarYRegistrar(clientes: int) : String + calcularComision(porcentaje: float) : double + calcularComision(porcentaje: int) : double + calcularPagoTotal(comision: double, diasTrabajados: int) : double

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **visitarClientes(clientes)**: retorna un texto con el nombre y el valor de clientes.
- **registrarVenta()**: retorna un texto con el nombre.
- **visitarYRegistrar(clientes)**: retorna visitarClientes(clientes) + registrarVenta(). Debe reutilizar los dos métodos anteriores.
- **calcularComision(porcentaje)**, versión float y versión int: retorna salarioBase * porcentaje / 100.
- **calcularPagoTotal(comision, diasTrabajados)**: retorna salarioBase / 30 * diasTrabajados + comision.

Main

- Crear un objeto con new Vendedor() y asignarle los 4 atributos con sus setters.
- Imprimir los 4 getters y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularComision(10) y calcularComision(10f).

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Vendedor.java** y **MainVendedor.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String nombre;
private String zona;
private double salarioBase;
private int ventasMes;

public Vendedor() {
}

public Vendedor(String nombre, String zona, double salarioBase, int ventasMes) {
    this.nombre = nombre;
    this.zona = ____;
    this.____ = salarioBase;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de nombre; escribe igual los de zona, salarioBase y ventasMes, cambiando el tipo.

```
public String getNombre() {
    return this.nombre;
}
public void setNombre(String nombre) {
    this.nombre = nombre;
}
```

3. Métodos de comportamiento. Retornan un texto; usa + para unir el texto con los atributos.

```
public String verDetalle() {
    return "Nombre: " + this.nombre + "\nZona: " + this.zona + ____ ;
}
public String visitarClientes(int clientes) {
    return "El vendedor " + this.nombre + " ... " + ____ ;
}
public String registrarVenta() {
    return "El vendedor " + ____ + " ..." ;
}
public String visitarYRegistrar(int clientes) {
    return this.visitarClientes(clientes) + " " + this.____();   // reutiliza los dos métodos
}
```

4. Métodos de cálculo. Las dos versiones son iguales; solo cambia el tipo del parámetro (float o int).

```
public double calcularComision(float porcentaje) {
    return this.salarioBase * porcentaje / 100;
}
public double calcularComision(int porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPagoTotal(double comision, int diasTrabajados) {
    // salarioBase / 30 * diasTrabajados + comision
    return ____ ;
}
```

5. Main. En otro archivo, MainVendedor.java:

```
public class MainVendedor {
    public static void main(String[] args) {
        Vendedor obj = new Vendedor();
        obj.setNombre("...");
        obj.setZona("...");
        obj.setSalarioBase(____);
        obj.setVentasMes(____);
        System.out.println(obj.getNombre());     // haz lo mismo con los otros 3 getters
        System.out.println(obj.verDetalle());
        System.out.println(obj.visitarClientes(5));
        System.out.println(obj.registrarVenta());
        System.out.println(obj.visitarYRegistrar(5));
        System.out.println(obj.calcularComision(10));       // versión int
        System.out.println(obj.calcularComision(10f));      // versión float
        System.out.println(obj.calcularPagoTotal(____, ____));
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Daniel Alejandro Guevara Rosas
Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Piloto (como el ejercicio Empleado)

Piloto
- nombre : String - aerolinea : String - valorHoraVuelo : double - horasVueloMes : int
+ Piloto() + Piloto(nombre: String, aerolinea: String, valorHoraVuelo: double, horasVueloMes: int) + getters y setters de los 4 atributos + verDetalle() : String + volar(horas: int) : String + revisarAvion() : String + revisarYVolar(horas: int) : String + calcularViatico(porcentaje: float) : double + calcularViatico(porcentaje: int) : double + calcularPagoVuelo(viatico: double, horas: int) : double

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **volar(horas)**: retorna un texto con el nombre y el valor de horas.
- **revisarAvion()**: retorna un texto con el nombre.
- **revisarYVolar(horas)**: retorna volar(horas) + revisarAvion(). Debe reutilizar los dos métodos anteriores.
- **calcularViatico(porcentaje)**, versión float y versión int: retorna valorHoraVuelo * porcentaje / 100.
- **calcularPagoVuelo(viatico, horas)**: retorna valorHoraVuelo * horas + viatico.

Main

- Crear un objeto con new Piloto() y asignarle los 4 atributos con sus setters.
- Imprimir los 4 getters y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularViatico(10) y calcularViatico(10f).

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Piloto.java** y **MainPiloto.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String nombre;
private String aerolinea;
private double valorHoraVuelo;
private int horasVueloMes;

public Piloto() {
}

public Piloto(String nombre, String aerolinea, double valorHoraVuelo, int horasVueloMes) {
    this.nombre = nombre;
    this.aerolinea = ____;
    this.____ = valorHoraVuelo;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de nombre; escribe igual los de aerolinea, valorHoraVuelo y horasVueloMes, cambiando el tipo.

```
public String getNombre() {
    return this.nombre;
}
public void setNombre(String nombre) {
    this.nombre = nombre;
}
```

3. Métodos de comportamiento. Retornan un texto; usa + para unir el texto con los atributos.

```
public String verDetalle() {
    return "Nombre: " + this.nombre + "\nAerolinea: " + this.aerolinea + ____ ;
}
public String volar(int horas) {
    return "El piloto " + this.nombre + " ... " + ____ ;
}
public String revisarAvion() {
    return "El piloto " + ____ + " ..." ;
}
public String revisarYVolar(int horas) {
    return this.volar(horas) + " " + this.____();   // reutiliza los dos métodos
}
```

4. Métodos de cálculo. Las dos versiones son iguales; solo cambia el tipo del parámetro (float o int).

```
public double calcularViatico(float porcentaje) {
    return this.valorHoraVuelo * porcentaje / 100;
}
public double calcularViatico(int porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPagoVuelo(double viatico, int horas) {
    // valorHoraVuelo * horas + viatico
    return ____ ;
}
```

5. Main. En otro archivo, MainPiloto.java:

```
public class MainPiloto {
    public static void main(String[] args) {
        Piloto obj = new Piloto();
        obj.setNombre("...");
        obj.setAerolinea("...");
        obj.setValorHoraVuelo(____);
        obj.setHorasVueloMes(____);
        System.out.println(obj.getNombre());     // haz lo mismo con los otros 3 getters
        System.out.println(obj.verDetalle());
        System.out.println(obj.volar(5));
        System.out.println(obj.revisarAvion());
        System.out.println(obj.revisarYVolar(5));
        System.out.println(obj.calcularViatico(10));       // versión int
        System.out.println(obj.calcularViatico(10f));      // versión float
        System.out.println(obj.calcularPagoVuelo(____, ____));
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Santiago Muñoz Eslava

Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Bibliotecario (como el ejercicio Empleado)

Bibliotecario
- nombre : String - sede : String - salarioBase : double - aniosServicio : int
+ Bibliotecario() + Bibliotecario(nombre: String, sede: String, salarioBase: double, aniosServicio: int) + getters y setters de los 4 atributos + verDetalle() : String + prestarLibros(libros: int) : String + organizarEstantes() : String + prestarYOrganizar(libros: int) : String + calcularBono(porcentaje: float) : double + calcularBono(porcentaje: int) : double + calcularPagoMes(bono: double, dias: int) : double

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **prestarLibros(libros)**: retorna un texto con el nombre y el valor de libros.
- **organizarEstantes()**: retorna un texto con el nombre.
- **prestarYOrganizar(libros)**: retorna prestarLibros(libros) + organizarEstantes(). Debe reutilizar los dos métodos anteriores.
- **calcularBono(porcentaje)**, versión float y versión int: retorna salarioBase * porcentaje / 100.
- **calcularPagoMes(bono, dias)**: retorna salarioBase / 30 * dias + bono.

Main

- Crear un objeto con new Bibliotecario() y asignarle los 4 atributos con sus setters.
- Imprimir los 4 getters y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularBono(10) y calcularBono(10f).

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Bibliotecario.java** y **MainBibliotecario.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String nombre;
private String sede;
private double salarioBase;
private int aniosServicio;

public Bibliotecario() {
}

public Bibliotecario(String nombre, String sede, double salarioBase, int aniosServicio) {
    this.nombre = nombre;
    this.sede = ____;
    this.____ = salarioBase;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de nombre; escribe igual los de sede, salarioBase y aniosServicio, cambiando el tipo.

```
public String getNombre() {
    return this.nombre;
}
public void setNombre(String nombre) {
    this.nombre = nombre;
}
```

3. Métodos de comportamiento. Retornan un texto; usa + para unir el texto con los atributos.

```
public String verDetalle() {
    return "Nombre: " + this.nombre + "\nSede: " + this.sede + ____ ;
}
public String prestarLibros(int libros) {
    return "El bibliotecario " + this.nombre + " ... " + ____ ;
}
public String organizarEstantes() {
    return "El bibliotecario " + ____ + " ..." ;
}
public String prestarYOrganizar(int libros) {
    return this.prestarLibros(libros) + " " + this.____();   // reutiliza los dos métodos
}
```

4. Métodos de cálculo. Las dos versiones son iguales; solo cambia el tipo del parámetro (float o int).

```
public double calcularBono(float porcentaje) {
    return this.salarioBase * porcentaje / 100;
}
public double calcularBono(int porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPagoMes(double bono, int dias) {
    // salarioBase / 30 * dias + bono
    return ____ ;
}
```

5. Main. En otro archivo, MainBibliotecario.java:

```
public class MainBibliotecario {
    public static void main(String[] args) {
        Bibliotecario obj = new Bibliotecario();
        obj.setNombre("...");
        obj.setSede("...");
        obj.setSalarioBase(____);
        obj.setAniosServicio(____);
        System.out.println(obj.getNombre());     // haz lo mismo con los otros 3 getters
        System.out.println(obj.verDetalle());
        System.out.println(obj.prestarLibros(5));
        System.out.println(obj.organizarEstantes());
        System.out.println(obj.prestarYOrganizar(5));
        System.out.println(obj.calcularBono(10));       // versión int
        System.out.println(obj.calcularBono(10f));      // versión float
        System.out.println(obj.calcularPagoMes(____, ____));
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: María Fernanda Rodríguez Montero
Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase GuiaTuristico (como el ejercicio Empleado)

GuiaTuristico
- nombre : String - idioma : String - valorRecorrido : double - aniosExperiencia : int
+ GuiaTuristico() + GuiaTuristico(nombre: String, idioma: String, valorRecorrido: double, aniosExperiencia: int) + getters y setters de los 4 atributos + verDetalle() : String + guiarGrupo(personas: int) : String + explicarHistoria() : String + guiarYExplicar(personas: int) : String + calcularPropina(porcentaje: float) : double + calcularPropina(porcentaje: int) : double + calcularPagoDia(propina: double, recorridos: int) : double

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **guiarGrupo(personas)**: retorna un texto con el nombre y el valor de personas.
- **explicarHistoria()**: retorna un texto con el nombre.
- **guiarYExplicar(personas)**: retorna guiarGrupo(personas) + explicarHistoria(). Debe reutilizar los dos métodos anteriores.
- **calcularPropina(porcentaje)**, versión float y versión int: retorna valorRecorrido * porcentaje / 100.
- **calcularPagoDia(propina, recorridos)**: retorna (valorRecorrido + propina) * recorridos.

Main

- Crear un objeto con new GuiaTuristico() y asignarle los 4 atributos con sus setters.
- Imprimir los 4 getters y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularPropina(10) y calcularPropina(10f).

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **GuiaTuristico.java** y **MainGuiaTuristico.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String nombre;
private String idioma;
private double valorRecorrido;
private int aniosExperiencia;

public GuiaTuristico() {
}

public GuiaTuristico(String nombre, String idioma, double valorRecorrido, int aniosExperiencia) {
    this.nombre = nombre;
    this.idioma = ____;
    this.____ = valorRecorrido;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de nombre; escribe igual los de idioma, valorRecorrido y aniosExperiencia, cambiando el tipo.

```
public String getNombre() {
    return this.nombre;
}
public void setNombre(String nombre) {
    this.nombre = nombre;
}
```

3. Métodos de comportamiento. Retornan un texto; usa + para unir el texto con los atributos.

```
public String verDetalle() {
    return "Nombre: " + this.nombre + "\nIdioma: " + this.idioma + ____ ;
}
public String guiarGrupo(int personas) {
    return "El guiaturistico " + this.nombre + " ... " + ____ ;
}
public String explicarHistoria() {
    return "El guiaturistico " + ____ + " ..." ;
}
public String guiarYExplicar(int personas) {
    return this.guiarGrupo(personas) + " " + this.____();   // reutiliza los dos métodos
}
```

4. Métodos de cálculo. Las dos versiones son iguales; solo cambia el tipo del parámetro (float o int).

```
public double calcularPropina(float porcentaje) {
    return this.valorRecorrido * porcentaje / 100;
}
public double calcularPropina(int porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPagoDia(double propina, int recorridos) {
    // (valorRecorrido + propina) * recorridos
    return ____ ;
}
```

5. Main. En otro archivo, MainGuiaTuristico.java:

```
public class MainGuiaTuristico {
    public static void main(String[] args) {
        GuiaTuristico obj = new GuiaTuristico();
        obj.setNombre("...");
        obj.setIdioma("...");
        obj.setValorRecorrido(____);
        obj.setAniosExperiencia(____);
        System.out.println(obj.getNombre());     // haz lo mismo con los otros 3 getters
        System.out.println(obj.verDetalle());
        System.out.println(obj.guiarGrupo(5));
        System.out.println(obj.explicarHistoria());
        System.out.println(obj.guiarYExplicar(5));
        System.out.println(obj.calcularPropina(10));       // versión int
        System.out.println(obj.calcularPropina(10f));      // versión float
        System.out.println(obj.calcularPagoDia(____, ____));
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Sairo Luis Izquierdo Vásquez
Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase FuncionCine (como el ejercicio Producto)

FuncionCine
- pelicula : String - sala : String - valorEntrada : double - sillasLibres : int
+ FuncionCine() + FuncionCine(pelicula: String, sala: String, valorEntrada: double, sillasLibres: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + comprarEntradas(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna valorEntrada * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna valorEntrada - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **comprarEntradas(cantidad)**: si cantidad es menor o igual a sillasLibres, resta cantidad a sillasLibres y retorna "Operación realizada. Quedan X". Si no, no cambia sillasLibres y retorna "No hay suficientes sillasLibres".

Main

- Crear dos objetos: uno con new FuncionCine() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar comprarEntradas dos veces: una con cantidad suficiente y otra con una cantidad mayor a sillasLibres.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **FuncionCine.java** y **MainFuncionCine.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String pelicula;
private String sala;
private double valorEntrada;
private int sillasLibres;

public FuncionCine() {
}

public FuncionCine(String pelicula, String sala, double valorEntrada, int sillasLibres) {
    this.pelicula = pelicula;
    this.sala = ____;
    this.____ = valorEntrada;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de pelicula; escribe igual los de sala, valorEntrada y sillasLibres, cambiando el tipo.

```
public String getPelicula() {
    return this.pelicula;
}
public void setPelicula(String pelicula) {
    this.pelicula = pelicula;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "Pelicula: " + this.pelicula + "\nSala: " + this.sala + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.valorEntrada * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.valorEntrada - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. comprarEntradas. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia sillasLibres.

```
public String comprarEntradas(int cantidad) {
    if (cantidad <= this.____) {
        this.sillasLibres = this.sillasLibres - ____;
        return "Operación realizada. Quedan " + this.sillasLibres;
    } else {
        return "No hay suficientes sillasLibres";
    }
}
```

5. Main. En otro archivo, MainFuncionCine.java:

```
public class MainFuncionCine {
    public static void main(String[] args) {
        FuncionCine obj1 = new FuncionCine();
        obj1.setPelicula("...");
        obj1.setSala("...");
        obj1.setValorEntrada(____);
        obj1.setSillasLibres(10);
        FuncionCine obj2 = new FuncionCine("...", "...", ____, ____);

        System.out.println(obj1.getPelicula());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.comprarEntradas(3));    // sí alcanza
        System.out.println(obj1.comprarEntradas(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Juan David Lara Bernal

Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Hotel (como el ejercicio Producto)

Hotel
- nombreHotel : String - ciudad : String - valorNoche : double - habitacionesLibres : int
+ Hotel() + Hotel(nombreHotel: String, ciudad: String, valorNoche: double, habitacionesLibres: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + reservarHabitaciones(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna valorNoche * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna valorNoche - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **reservarHabitaciones(cantidad)**: si cantidad es menor o igual a habitacionesLibres, resta cantidad a habitacionesLibres y retorna "Operación realizada. Quedan X". Si no, no cambia habitacionesLibres y retorna "No hay suficientes habitacionesLibres".

Main

- Crear dos objetos: uno con new Hotel() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar reservarHabitaciones dos veces: una con cantidad suficiente y otra con una cantidad mayor a habitacionesLibres.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Hotel.java** y **MainHotel.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String nombreHotel;
private String ciudad;
private double valorNoche;
private int habitacionesLibres;

public Hotel() {
}

public Hotel(String nombreHotel, String ciudad, double valorNoche, int habitacionesLibres) {
    this.nombreHotel = nombreHotel;
    this.ciudad = ____;
    this.____ = valorNoche;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de nombreHotel; escribe igual los de ciudad, valorNoche y habitacionesLibres, cambiando el tipo.

```
public String getNombreHotel() {
    return this.nombreHotel;
}
public void setNombreHotel(String nombreHotel) {
    this.nombreHotel = nombreHotel;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "NombreHotel: " + this.nombreHotel + "\nCiudad: " + this.ciudad + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.valorNoche * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.valorNoche - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. reservarHabitaciones. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia habitacionesLibres.

```
public String reservarHabitaciones(int cantidad) {
    if (cantidad <= this.____) {
        this.habitacionesLibres = this.habitacionesLibres - ____;
        return "Operación realizada. Quedan " + this.habitacionesLibres;
    } else {
        return "No hay suficientes habitacionesLibres";
    }
}
```

5. Main. En otro archivo, MainHotel.java:

```
public class MainHotel {
    public static void main(String[] args) {
        Hotel obj1 = new Hotel();
        obj1.setNombreHotel("...");
        obj1.setCiudad("...");
        obj1.setValorNoche(____);
        obj1.setHabitacionesLibres(10);
        Hotel obj2 = new Hotel("...", "...", ____, ____);

        System.out.println(obj1.getNombreHotel());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.reservarHabitaciones(3));    // sí alcanza
        System.out.println(obj1.reservarHabitaciones(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Jorjan Oviedo Álvarez

Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase Repuesto (como el ejercicio Producto)

Repuesto
- nombre : String - vehiculo : String - precio : double - existencias : int
+ Repuesto() + Repuesto(nombre: String, vehiculo: String, precio: double, existencias: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + despacharRepuestos(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna precio * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna precio - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **despacharRepuestos(cantidad)**: si cantidad es menor o igual a existencias, resta cantidad a existencias y retorna "Operación realizada. Quedan X". Si no, no cambia existencias y retorna "No hay suficientes existencias".

Main

- Crear dos objetos: uno con new Repuesto() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar despacharRepuestos dos veces: una con cantidad suficiente y otra con una cantidad mayor a existencias.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **Repuesto.java** y **MainRepuesto.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String nombre;
private String vehiculo;
private double precio;
private int existencias;

public Repuesto() {
}

public Repuesto(String nombre, String vehiculo, double precio, int existencias) {
    this.nombre = nombre;
    this.vehiculo = ____;
    this.____ = precio;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de nombre; escribe igual los de vehiculo, precio y existencias, cambiando el tipo.

```
public String getNombre() {
    return this.nombre;
}
public void setNombre(String nombre) {
    this.nombre = nombre;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "Nombre: " + this.nombre + "\nVehiculo: " + this.vehiculo + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.precio * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.precio - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. despacharRepuestos. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia existencias.

```
public String despacharRepuestos(int cantidad) {
    if (cantidad <= this.____) {
        this.existencias = this.existencias - ____;
        return "Operación realizada. Quedan " + this.existencias;
    } else {
        return "No hay suficientes existencias";
    }
}
```

5. Main. En otro archivo, MainRepuesto.java:

```
public class MainRepuesto {
    public static void main(String[] args) {
        Repuesto obj1 = new Repuesto();
        obj1.setNombre("...");
        obj1.setVehiculo("...");
        obj1.setPrecio(____);
        obj1.setExistencias(10);
        Repuesto obj2 = new Repuesto("...", "...", ____, ____);

        System.out.println(obj1.getNombre());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.despacharRepuestos(3));    // sí alcanza
        System.out.println(obj1.despacharRepuestos(50));   // no alcanza
    }
}
```

Recuperación segundo corte

Programación Orientada a Objetos · NRC 60-80980 · UNIMINUTO · Tiempo: 90 minutos

Estudiante: Nicole Julay Ramírez Silva

Debe entregar **un código**: la clase y el main en archivos separados, en la carpeta **Recuperacion_Corte2** de su repositorio de GitHub, antes de terminar la clase. Se califica con la misma rúbrica del ejercicio original; si no compila, no se califica.

Ejercicio de recuperación: clase PasajeBus (como el ejercicio Producto)

PasajeBus
- ruta : String - empresa : String - valorPasaje : double - sillasDisponibles : int
+ PasajeBus() + PasajeBus(ruta: String, empresa: String, valorPasaje: double, sillasDisponibles: int) + getters y setters de los 4 atributos + verDetalle() : String + calcularDescuento(porcentaje: int) : double + calcularDescuento(porcentaje: double) : double + calcularPrecioFinal(porcentaje: double) : double + venderPasajes(cantidad: int) : String

Qué debe hacer cada método

- **verDetalle()**: retorna un texto con los 4 atributos.
- **calcularDescuento(porcentaje)**, versión int y versión double: retorna valorPasaje * porcentaje / 100.0
- **calcularPrecioFinal(porcentaje)**: retorna valorPasaje - calcularDescuento(porcentaje). Debe reutilizar calcularDescuento.
- **venderPasajes(cantidad)**: si cantidad es menor o igual a sillasDisponibles, resta cantidad a sillasDisponibles y retorna "Operación realizada. Quedan X". Si no, no cambia sillasDisponibles y retorna "No hay suficientes sillasDisponibles".

Main

- Crear dos objetos: uno con new PasajeBus() y setters, y otro con el constructor de 4 parámetros.
- Imprimir los 4 getters de un objeto y llamar todos los métodos al menos una vez.
- Probar la sobrecarga: calcularDescuento(10) y calcularDescuento(12.5).
- Probar venderPasajes dos veces: una con cantidad suficiente y otra con una cantidad mayor a sillasDisponibles.

GitHub: carpeta **Recuperacion_Corte2**, con los archivos **PasajeBus.java** y **MainPasajeBus.java**.

Ayudas para escribir el código (completa los espacios ____)

1. Atributos y constructores. this.atributo es el espacio del objeto; el nombre sin this es el dato que llega.

```
private String ruta;
private String empresa;
private double valorPasaje;
private int sillasDisponibles;

public PasajeBus() {
}

public PasajeBus(String ruta, String empresa, double valorPasaje, int sillasDisponibles) {
    this.ruta = ruta;
    this.empresa = ____;
    this.____ = valorPasaje;
    this.____ = ____;
}
```

2. Getter y setter. Este es el par de ruta; escribe igual los de empresa, valorPasaje y sillasDisponibles, cambiando el tipo.

```
public String getRuta() {
    return this.ruta;
}
public void setRuta(String ruta) {
    this.ruta = ruta;
}
```

3. verDetalle y descuentos. Las dos versiones de calcularDescuento son iguales; solo cambia el tipo del parámetro.

```
public String verDetalle() {
    return "Ruta: " + this.ruta + "\nEmpresa: " + this.empresa + ____ ;
}
public double calcularDescuento(int porcentaje) {
    return this.valorPasaje * porcentaje / 100.0;
}
public double calcularDescuento(double porcentaje) {
    return ____ ;          // la misma fórmula
}
public double calcularPrecioFinal(double porcentaje) {
    return this.valorPasaje - this.____(porcentaje);   // reutiliza calcularDescuento
}
```

4. venderPasajes. Primero compara; solo si alcanza, resta. Si no alcanza, no cambia sillasDisponibles.

```
public String venderPasajes(int cantidad) {
    if (cantidad <= this.____) {
        this.sillasDisponibles = this.sillasDisponibles - ____;
        return "Operación realizada. Quedan " + this.sillasDisponibles;
    } else {
        return "No hay suficientes sillasDisponibles";
    }
}
```

5. Main. En otro archivo, MainPasajeBus.java:

```
public class MainPasajeBus {
    public static void main(String[] args) {
        PasajeBus obj1 = new PasajeBus();
        obj1.setRuta("...");
        obj1.setEmpresa("...");
        obj1.setValorPasaje(____);
        obj1.setSillasDisponibles(10);
        PasajeBus obj2 = new PasajeBus("...", "...", ____, ____);

        System.out.println(obj1.getRuta());   // haz lo mismo con los otros 3 getters
        System.out.println(obj1.verDetalle());
        System.out.println(obj1.calcularDescuento(10));     // versión int
        System.out.println(obj1.calcularDescuento(12.5));   // versión double
        System.out.println(obj1.calcularPrecioFinal(10));
        System.out.println(obj1.venderPasajes(3));    // sí alcanza
        System.out.println(obj1.venderPasajes(50));   // no alcanza
    }
}
```